

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №5
з курсу «Хмарні технології та великі дані»
на тему:
«Time-out (Тайм-аут)»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи.....	4
Теоретичні відомості	4
Структура програмної реалізації	5
Тестування програмної реалізації.....	5
Висновок	8
Код програми	9

Тема

Обмеження максимального часу очікування асинхронних операцій та запобігання витоку ресурсів за допомогою патерну Time-out (Тайм-аут).

Мета

Дослідження принципу функціонування патерну Time-out (Тайм-аут). Розробка надійної типізованої обгортки над асинхронними операціями мовою TypeScript для контролю дедлайнів виконання із підтримкою кооперативного переривання побічних ефектів через AbortSignal, очищенням активних таймерів Node.js, контрольованим комбінуванням із патерном Retry та забезпеченням можливості автономної ручної перевірки (через демонстраційний скрипт та HTTP-сервер).

Вхідні дані

Параметр	Значення	Призначення
timeoutMs	Додатне число в мілісекундах (наприклад, 50, 100, 200)	Граничний інтервал очікування, після якого операція переривається.
signal	Екземпляр AbortSignal	Інструмент кооперативного скасування тривалих операцій та зупинки фонових побічних ефектів.
retries	Ціле невід'ємне число (наприклад, 2)	Кількість повторних спроб виконання операції у разі виникнення помилки або тайм-ауту.
delayMs	Число в мс (за замовчуванням 0)	Пауза між повторними спробами виконання операції.
taskMs	Число в мс (наприклад, 25, 500)	Тривалість імітованого навантаження / виконання задачі при ручному тестуванні

Завдання

- Розробити універсальну функцію вищого порядку withTimeout<T>(fn, timeoutMs) мовою TypeScript зі строгим збереженням типів результату Promise<T> та валідацією вхідного дедлайну.
- Реалізувати власний типізований клас помилки TimeoutError, що фіксує перевищений ліміт мілісекунд.
- Забезпечити підтримку механізму кооперативного переривання виконання за допомогою стандартного AbortController / AbortSignal, щоб запобігти продовженню виконання побічних ефектів фоновій задачі після настання тайм-ауту.

- Гарантувати коректне очищення таймерів через блок `finally (clearTimeout)`, запобігаючи блокуванню Event Loop та витокам пам'яті.
- Реалізувати допоміжну обгортку `withRetry(fn, options)` та перевірити коректність їхнього комбінування без безконтрольного множення загального часу очікування.
- Реалізувати середовище автономної ручної перевірки без сторонніх тестових бібліотек (через консольний демонстраційний скрипт та HTTP-сервер з маршрутами перевірки через `curl`).

Критерії виконання роботи

- *Успішне завершення до вичерпання дедлайну:* якщо операція повертає результат швидше за `timeoutMs`, `withTimeout` успішно резолвить значення без затримок, а запланований таймер дедлайну негайно очищається.
- *Перевищення ліміту часу:* якщо операція триває довше, ніж `timeoutMs`, проміс негайно відхиляється екземпляром помилки `TimeoutError` із точним зазначенням встановленого ліміту.
- *Зупинка побічних ефектів:* при настанні тайм-ауту викликається `controller.abort()`, що сповіщає підписників `signal.addEventListener('abort', ...)` та припиняє будь-яку подальшу активність (наприклад, `clearInterval`).
- *Контрольоване комбінування з Retry:* повторні спроби виконуються суворо згідно з лімітом спроб (`retries + 1`), тайм-аут контролює кожен окрему спробу, а помилка `TimeoutError` коректно передається нагору після вичерпання всіх спроб.
- *Автономність перевірки:* наявність ізольованих інструментів перевірки поведінки через термінал (консольні логи) та зовнішній HTTP API (статуси 200 OK vs 504 Gateway Timeout).

Теоретичні відомості

Time-out (Тайм-аут) — це поведінковий патерн проектування та забезпечення відмовостійкості (Resilience Pattern) розподілених систем, метою якого є обмеження максимального часу, протягом якого клієнт або компонент очікує на завершення асинхронної операції чи відповіді від стороннього сервісу.

У сучасних асинхронних середовищах відсутність тайм-ауту створює загрозу вичерпання пулу з'єднань, зависання черг подій та каскадних збоїв (cascading failures). Для реалізації тайм-ауту в JavaScript/TypeScript зазвичай використовується конкурентне виконання через `Promise.race`, де власне задача змагається з штучно створеним таймером очікування.

Ключовим аспектом надійного тайм-ауту є скасування операції (Cancellation). Відхилення промісу за тайм-аутом у JavaScript саме по собі не зупиняє виконання коду всередині асинхронної функції. Якщо функція продовжує записувати дані у сховище чи виконувати мережеві запити після виходу за тайм-аут, виникають паразитні побічні ефекти. Для вирішення цієї

проблеми застосовується стандарт `AbortController` / `AbortSignal`, який дозволяє слухачам усередині операції отримати сигнал переривання, своєчасно очистити дескриптори ресурсів (наприклад, таймери, сокети) та припинити виконання.

При комбінуванні патернів `Time-out` та `Retry` важливо враховувати архітектурний рівень застосування дедлайну:

- *Per-attempt timeout*: тайм-аут застосовується до кожної окремої спроби, гарантуючи, що жоден завислий запит не заблокує процес довше заданого інтервалу.
- *Global timeout*: загальний ліміт часу на весь ланцюжок спроб включно з паузами між ними (`delayMs`), що запобігає неконтрольованому множенню загального часу очікування.

Структура програмної реалізації

Рішення складається з таких модулів:

- *src/timeout.ts* — ядро патерну: клас винятку `TimeoutError`, функція `withTimeout` із генерацією `AbortController`, конкурентним змаганням через `Promise.race` та очищенням таймерів у блоці `finally`.
- *src/retry.ts* — допоміжна утиліта `withRetry` для повторних спроб асинхронних операцій з регульованою кількістю спроб (`retries`) та затримкою (`delayMs`).
- *manual-test.ts* — автономний консольний скрипт, що наочно демонструє роботу всіх трьох ключових сценаріїв із логуванням стану побічних ефектів.
- *server.ts* — демонстраційний HTTP-сервер, що реалізує ендпоінти для ручної перевірки поведінки мережевими HTTP-запитами:
 - `GET /timeout/fast` — швидка задача, повертає HTTP 200.
 - `GET /timeout/slow` — тривала задача, повертає HTTP 504 Gateway Timeout та скасовує фонову роботу.
 - `GET /retry` — комбінація тайм-ауту та повторних спроб, демонструє обмежену кількість спроб перед поверненням HTTP 504

Тестування програмної реалізації

Перевірка коректності реалізованого патерну виконувалася двома взаємодоповнюючими способами: консольним скриптом ручної верифікації та через зовнішні HTTP-запити.

1. Автономний консольний запуск (*manual-test.ts*)

Запуск скрипта ручної верифікації виконується командою:

```

$ npm run manual

> timeout-pattern-lab@1.0.0 manual
> tsx manual-test.ts

=== Case 1: fast task ===
[OK] fast task completed

=== Case 2: slow task and cancellation ===
slow task still running
slow task still running
side effect cancelled by AbortSignal
[TIMEOUT] Operation timed out after 60 ms
[CHECK] side effect active: false

=== Case 3: retrying a slow task with a timeout
per attempt ===
attempt 1: running (60 ms timeout)
attempt 2: running (60 ms timeout)
attempt 3: running (60 ms timeout)
[STOPPED] Operation timed out after 60 ms
[CHECK] bounded attempts: 3

```

Рис.1 – Автономний консольний запуск патерну time-out

Результати виконання сценаріїв:

- У Case 1 операція тривалістю 25 мс успішно вклалася у вікно 100 мс.
- У Case 2 після настання тайм-ауту (60 мс) спрацював AbortSignal, який зупинив нескінченний інтервал (clearInterval), запобігши подальшим логам у консоль, а прапорець sideEffectActive скинувся у false.
- У Case 3 операція була повторена рівно 3 рази (початкова спроба + 2 ретраї), кожна спроба тривала суворо не більше 60 мс, підтверджуючи прогнозованість загального часу очікування.

2. Ручна перевірка через HTTP-сервер

Запуск демонстраційного сервера:

```

$ npm run server

> timeout-pattern-lab@1.0.0 server
> tsx server.ts

Timeout demo server listening at http://localhost:3000

```

Рис.2 – Старт сервера

Перевірка швидкого ендпоінту (/timeout/fast):

```

$ curl "http://localhost:3000/timeout/fast?taskMs=25&timeoutMs=200"
{"ok":true,"result":"task completed in 25 ms","timeoutMs":200,"taskMs":25}

```

Рис.3 – Перевірка швидкого ендпоінту

Результат: HTTP-статус: 200 ОК. Задача встигла виконатися до дедлайну.

Перевірка перевищення тайм-ауту та скасування (/timeout/slow):

```
$ curl -i "http://localhost:3000/timeout/slow?taskMs=500&timeoutMs=50"
HTTP/1.1 504 Gateway Timeout
content-type: application/json
Date: Sun, 20 Sep 2026 16:46:21 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"ok":false,"timedOut":true,"cancelled":true,"error":"TimeoutError: Operation timed out after 50 ms","timeoutMs":50,"taskMs":500}
```

Рис.4 – Перевірка перевищення тайм-ауту

Результат: Сервер негайно повернув помилку 504 Gateway Timeout після 50 мс, не чекаючи 500 мс, а фоновий таймер задачі був очищений за сигналом скасування.

Перевірка зв'язки з Retry (/retry):

```
$ curl -i "http://localhost:3000/retry?taskMs=500&timeoutMs=50&retries=2"
HTTP/1.1 504 Gateway Timeout
content-type: application/json
Date: Sun, 20 Sep 2026 16:47:17 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

{"ok":false,"timedOut":true,"attempts":3,"timeoutMs":50,"taskMs":500,"retries":2,"error":"TimeoutError: Operation timed out after 50 ms"}
```

Рис.5 – Перевірка зв'язки з Retry

Результат: HTTP-статус: 504 Gateway Timeout. Сервіс виконав суворо 3 спроби по 50 мс кожна і коректно завершив запит.

Висновок

У ході виконання лабораторної роботи було досліджено теоретичні засади та практичні аспекти реалізації патерну Time-out (Тайм-аут) для асинхронного середовища TypeScript.

Було розроблено надійну обгортку `withTimeout`, яка ефективно вирішує проблему «завислих» асинхронних операцій шляхом конкурентного обмеження часу очікування. Інтеграція стандартного механізму `AbortController` / `AbortSignal` дозволила реалізувати кооперативне скасування побічних ефектів, що унеможливорює витік пам'яті та холосту роботу процесора після переривання дедлайну. Завдяки обов'язковому очищенню активних таймерів у секції `finally`, код не утримує чергу подій `Node.js`.

Комбінування `withTimeout` із механізмом повторних спроб `withRetry` підтвердило можливість побудови стійких (resilient) архітектурних рішень із передбачуваним верхнім лімітом часу виконання. Усі закладені сценарії було повністю підтверджено консольною демонстрацією та перевіркою мережевими викликами HTTP API.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_5 та має повну інструкцію запуску даної програми.